

PROJECT BASED EXPERIMENTAL LEARNING

COURSE – NOSQL

PROJECT REPORT TOPIC – GADGET STORE INVENTORY MANAGEMENT SYSTEM USING MONGODB

SUBMITTED TO
Mr. HARENDRA SINGH

SUBMITTED BY
ASTIK TIWARI
BCA (DS & AI)
ROLL NO. – 1250258151
SCHOOL OF COMPUTER APPLICATIONS
BABU BANARASI DAS UNIVERSITY, LUCKNOW

ACADEMIC SESSION 2026–2027

INDEX

Sr. No.	Section
1	Acknowledgement
2	Abstract
3	Introduction and Objectives
4	Methodology
5	Implementation Details
6	MongoDB Operations Demonstrated
7	Results
8	Conclusion and Future Scope
9	References

REPORT BASIS

This report presents the practical work carried out for the NoSQL course through a Gadget Store Inventory Management System built on MongoDB, a document-oriented NoSQL database.

ACKNOWLEDGEMENT

I sincerely express my gratitude to my teacher Mr. Harendra Singh for giving me the opportunity to work on this NoSQL project titled “Gadget Store Inventory Management System Using MongoDB”.

This project helped me understand the basic concepts of NoSQL databases and learn how MongoDB is used to create, retrieve, update, sort and delete data. I also learned how query operators such as greater than, less than, equal to, AND and OR are used to filter product records by price and category.

I am thankful to my teacher for the guidance and support provided throughout the project, and I acknowledge the MongoDB documentation and tools that helped me practise queries and understand document-based data management.

— ASTIK TIWARI

ABSTRACT

This report presents a Gadget Store Inventory Management System developed using MongoDB, a NoSQL document-oriented database. The project shows how product information of an electronics store can be stored as documents and managed through MongoDB queries.

The system uses a GadgetStore database and a products collection containing 50 product records across five categories: Laptops, Smartphones, Accessories, Audio and Wearables. Each record has a product ID, name, category, price and stock quantity. The project demonstrates database and collection creation, insertion, retrieval, filtering, updating, deletion, sorting, limiting, category-based searching and document counting.

Comparison operators (\$eq, \$gt, \$lt, \$gte, \$lte) and logical operators (\$and, \$or) are also demonstrated, giving practical exposure to core NoSQL operations.

KEY FEATURES

- Document-based product data storage using MongoDB.
- Creation of a database and collection.
- Insertion of 50 product documents.
- CRUD operations: create, read, update and delete.
- Comparison and logical query operators on price and category.
- Sorting, limiting, category-based searching and counting.

INTRODUCTION

MongoDB is a NoSQL database management system that stores information as flexible documents instead of fixed rows and columns. This makes it well suited to applications such as product catalogues, where records may differ in structure and need to be queried quickly.

In this project an inventory system for a gadget store has been created using MongoDB. The database holds 50 products with their ID, name, category, price and stock. A series of operations is performed on these records to demonstrate practical database management, including filtering by price, combining conditions, updating stock levels, removing discontinued items, ordering by price, and counting what remains.

OBJECTIVES

- To understand the concept of NoSQL databases.
- To learn the basic features and document model of MongoDB.
- To create a database and collection in MongoDB.
- To insert and manage 50 product records.
- To perform CRUD operations on product documents.
- To use comparison operators \$eq, \$gt, \$lt, \$gte and \$lte.
- To perform logical AND and OR operations.
- To sort, limit, search and count documents.

PROJECT DATA MODEL

Field	Example	Purpose
productid	201	Unique product ID
name	Dell Inspiron 15	Name of the product
category	Laptops	Product category
price	55000	Selling price in rupees
stock	12	Units available in the store

METHODOLOGY

The project follows a practical sequence of MongoDB operations. The database is first selected, followed by collection creation and insertion of the product dataset. The stored records are then queried and manipulated using MongoDB operators.

Step	Activity	Description
1	Create Database	Create or switch to the GadgetStore database using use GadgetStore.
2	Create Collection	Create a products collection inside the database.
3	Insert Records	Insert 50 product documents with ID, name, category, price and stock.
4	Retrieve Data	Display the stored documents using find().
5	Filter Data	Use comparison operators to retrieve records based on price.
6	Update Data	Use updateOne() and updateMany() to modify stock.
7	Delete Data	Use deleteOne() and deleteMany() to remove records.
8	Logical Queries	Use \$and and \$or to combine conditions.
9	Organize Results	Use sort() and limit() to control query results.
10	Search and Count	Search by category and count documents in the collection.

BASIC DATABASE SETUP

```
use GadgetStore
db.createCollection("products")
```

The first command creates or switches to the GadgetStore database and the second creates the products collection.

IMPLEMENTATION DETAILS

The implementation is based on MongoDB shell (mongosh) syntax. The dataset contains 50 documents with product ID, name, category, price and stock, with 10 products in each of the five categories.

INSERTING PRODUCT RECORDS

```
db.products.insertMany([
  {productid:201, name:"Dell Inspiron 15", category:"Laptops", price:55000, stock:12},
  {productid:202, name:"HP Pavilion 14", category:"Laptops", price:62000, stock:8},
  {productid:203, name:"Lenovo IdeaPad Slim 3", category:"Laptops", price:48000, stock:15},
  ...
  {productid:250, name:"Mi Smart Band 7", category:"Wearables", price:3500, stock:36}
])
```

All 50 product documents are inserted successfully into the products collection.

DISPLAYING RECORDS

```
db.products.find()
```

The find() operation retrieves documents from the products collection. The shell displays records in batches of 20 documents.

COMPARISON OPERATORS

Operator	Example Query	Purpose / Matches
\$eq	db.products.find({ price: { \$eq: 1500 } })	Priced exactly 1500 (2 results)
\$gt	db.products.find({ price: { \$gt: 60000 } })	Priced above 60000 (7 results)
\$lt	db.products.find({ price: { \$lt: 1000 } })	Priced below 1000 (2 results)
\$gte	db.products.find({ price: { \$gte: 29000 } })	Priced 29000 or more (20 results)
\$lte	db.products.find({ price: { \$lte: 1500 } })	Priced 1500 or less (7 results)

MONGODB OPERATIONS DEMONSTRATED

UPDATE ONE

```
db.products.updateOne({ productid: 206 }, { $inc: { stock: 2 } })
```

Increases the stock of "MacBook Air M2" by two. The output shows matchedCount: 1 and modifiedCount: 1.

UPDATE MANY

```
db.products.updateMany({ category: "Accessories" }, { $inc: { stock: 10 } })
```

Adds 10 units of stock to every Accessories product. The output shows 10 documents matched and modified.

AND

```
db.products.find({ $and: [ { price: { $gte: 20000 } }, { category: "Smartphones" } ] })
```

Returns smartphones priced 20000 or more (8 results).

OR

```
db.products.find({ $or: [ { price: { $lte: 1000 } }, { category: "Wearables" } ] })
```

Returns products priced 1000 or less, or in the Wearables category (12 results).

SORT

```
db.products.find().sort({ price: 1 }).limit(5)
```

Sorts by price in ascending order and shows the five cheapest products: HDMI Cable 2m, Logitech Wireless Mouse, SanDisk 128GB Pen Drive, Noise Buds VS104, Fire-Boltt Ninja. Descending order (-1) shows the five costliest: Dell XPS 13, MacBook Air M2, HP Victus 16, iPhone 14, Lenovo ThinkPad E14.

LIMIT

```
db.products.find().limit(3)
```

Displays only the first three documents.

SEARCH BY CATEGORY

```
db.products.find({ category: "Audio" })
```

Finds all products in the Audio category (10 results).

DELETE ONE

```
db.products.deleteOne({ productid: 207 })
```

Deletes the single document with product ID 207 (Dell XPS 13).

DELETE MANY

```
db.products.deleteMany({ category: "Wearables" })
```

Deletes all Wearables documents; 10 documents are deleted.

COUNT

```
db.products.countDocuments({})
```

Counts documents currently in the collection; the final result is 39.

RESULTS

The project successfully demonstrated the core NoSQL operations required for managing a product inventory in MongoDB. The original dataset contained 50 product documents, and queries were executed to retrieve, filter, modify, delete and organise the records.

Operation	Result	Observation
Initial records inserted	50	50 product documents inserted successfully.
updateOne()	1 matched / 1 modified	Stock of product ID 206 increased by two.
updateMany()	10 matched / 10 modified	Stock increased by 10 for Accessories.
deleteOne()	1 deleted	Product ID 207 removed.
deleteMany()	10 deleted	Wearables documents removed.
Final count	39	Count reported after the deletion operations.

LEARNING OUTCOMES

- Understanding of the MongoDB document model and NoSQL approach.
- Practical use of database and collection creation commands.
- Hands-on experience with CRUD operations.
- Understanding of comparison and logical operators.
- Ability to sort, limit, search and count MongoDB documents.
- Understanding of how query results change after update and delete operations.

PROJECT RESULT

The project demonstrates a complete workflow for managing a gadget store inventory in MongoDB, from database creation and insertion through querying, updating, deletion and final document counting.

CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Gadget Store Inventory Management System containing product information was created and different MongoDB operations were performed on the data.

Through this project I learned how to insert, retrieve, update and delete documents, how to sort data in ascending and descending order, how comparison operators work, and how logical operators such as \$and and \$or combine conditions. Overall, the project helped me understand how MongoDB can be used to manage and query inventory data efficiently.

FUTURE SCOPE

- Develop a web-based inventory and billing interface connected to MongoDB.
- Add authentication and role-based access for store managers and staff.
- Add supplier records, purchase orders and automatic low-stock alerts.
- Add dashboards and charts for sales and stock analytics.
- Deploy the database using MongoDB Atlas and the application on a cloud platform.
- Add validation, indexing and aggregation queries for larger datasets.

REFERENCES

- MongoDB Documentation – MongoDB Query and CRUD concepts.
- MongoDB documentation on comparison and logical query operators.
- MongoDB documentation on update, delete, sort, limit and count operations.

THANK YOU!

Gadget Store Inventory Management System Using MongoDB

ASTIK TIWARI

BCA (DS & AI) | Roll No. 1250258151

Babu Banarasi Das University, Lucknow